

## EVALUACION CONJUNTA

### 1. Contexto

CavaLocal es un marketplace intermediario de vinos para Caracas. Actualmente cuenta con un **backend en NestJS** (API REST), un **frontend web** en vanilla JS y un **landing de marketing**. Dado el crecimiento del proyecto, se requiere implementar una trazabilidad completa mediante un **microservicio de auditoría** desacoplado, y un **dashboard en tiempo real** que visualice estos eventos mediante **Server-Sent Events (SSE)**.

El gran desafío de esta actividad es que todo el ecosistema (backend principal, microservicio de auditoría, base de datos, RabbitMQ y dashboard) debe ser **contenedorizado (Docker)** y **orquestrado en Kubernetes** para simular un entorno de producción escalable y resiliente.

### 2. Objetivo General

Diseñar, implementar, desplegar y orquestrar en **Kubernetes** un microservicio de auditoría que consuma eventos del backend principal vía **RabbitMQ**, los almacene, y exponga un **dashboard SSE** para monitorización en tiempo real.

### 3. Alcance

- **Backend (NestJS)**: Debe publicar mensajes en RabbitMQ por cada acción de escritura sobre las entidades del sistema (vinos, tiendas, usuarios, reservas, pagos).
- **Microservicio de Auditoría**: Consume los mensajes, almacena la traza (entidad, acción, usuario, fecha, data JSON) y expone una API REST para consultas.
- **Dashboard SSE**: Consume un endpoint SSE (expuesto por el microservicio o por un servicio adjunto) y actualiza la interfaz en tiempo real.
- **Despliegue**: Todo el sistema debe desplegarse en un clúster de Kubernetes (local con Minikube, Kind, o en la nube).

### 4. Requisitos Funcionales

#### 4.1. Microservicio de Auditoría

- **Registro**: Almacena entidad, acción, usuario (ID/email), timestamp y data JSON (antes/después en updates).
- **API de consulta**: Endpoints REST con paginación y filtros (por entidad, usuario, fecha, acción).
- **Consumo asíncrono**: Escucha la cola de RabbitMQ y confirma el mensaje solo tras persistir exitosamente (ACK manual).

#### 4.2. Dashboard en Tiempo Real (SSE)

- Visualiza la lista de auditorías recientes con actualización automática.
- Permite filtrar por entidad, acción y usuario.
- Muestra el detalle completo del JSON al hacer clic en un evento.
- **Reconexión automática** ante caídas del SSE.

#### 4.3. Integración con el Backend Actual (NestJS)

- Modificar los servicios (create/update/delete) para que **publiquen** un mensaje estructurado en el exchange audit.events de RabbitMQ.
- El mensaje debe incluir: entity, action, userId, userEmail, timestamp, data.
- Asegurar que el backend **no falle** si RabbitMQ no responde (usar try-catch, reconexión o publicaciones en buffer).

### 5. Requisitos Técnicos y de Infraestructura (Kubernetes con Ingress)

- **Orquestación obligatoria**: Todo el sistema debe ejecutarse sobre **Kubernetes**. Se recomienda **Minikube** o **Kind** para entornos locales, o un clúster en la nube.
- **Contenedores (Docker)**: Cada servicio debe tener su propio Dockerfile:
  - Backend NestJS.
  - Microservicio de auditoría (lenguaje libre: Go, Python, Node, Java, etc.).
  - Dashboard (puede ser servido por Nginx o un servidor Node).
- **Recursos de Kubernetes** (manifiestos YAML):
  - **Deployments** para cada servicio (con réplicas configurables).
  - **Services** (ClusterIP para comunicación interna entre pods).
  - **ConfigMaps** y **Secrets** para variables de entorno (credenciales de BD, RabbitMQ, JWT, etc.).
  - **StatefulSets** (opcional, recomendado) para bases de datos o RabbitMQ si se requiere persistencia.

- **PersistentVolumeClaims (PVC)** para los volúmenes de las bases de datos.
- **Ingress Controller y enrutamiento externo** (requisito obligatorio):
  - Se debe desplegar un **Ingress Controller** (ej. NGINX Ingress Controller) en el clúster.
  - Se debe crear un recurso **Ingress** que enrute el tráfico entrante hacia los diferentes servicios, utilizando el **dominio local** conjunta3p.espe.edu.ec.
  - Para que el dominio funcione en local, se deberá modificar el archivo /etc/hosts (o el equivalente en Windows) apuntando conjunta3p.espe.edu.ec a la IP del clúster (ej. IP de Minikube o del nodo).
  - Las rutas (paths) del Ingress deben definirse al menos para:
    - /api/audit → microservicio de auditoría (API REST).
    - /dashboard → dashboard web (SSE).
    - /api → backend principal de CavaLocal (opcional, pero valorable para centralizar todo el tráfico).
  - Se valorará el uso de **TLS/HTTPS** (con certificados autofirmados o cert-manager), aunque no es obligatorio.
- **Comunicación interna:** Los servicios deben resolverse por DNS interno de Kubernetes (ej. rabbitmq-service.default.svc.cluster.local).
- **Health Checks:** Implementar **readinessProbe** y **livenessProbe** en todos los Deployments.
- **Escalabilidad:** El microservicio de auditoría debe poder escalar horizontalmente (replicas: 2 o más) sin duplicar el consumo de mensajes (RabbitMQ maneja la cola competitiva).

## 6. Entregables

1. **Repositorio Git** con:
  - Código fuente del microservicio de auditoría.
  - Código fuente del dashboard.
  - Parches o modificaciones del backend de CavaLocal.
  - Carpeta /k8s con todos los manifiestos YAML (Deployments, Services, ConfigMaps, Secrets, PVC, Ingress, etc.).
  - Archivos Dockerfile para cada componente.
  - Scripts de ayuda (ej. deploy.sh, destroy.sh).
2. **Documentación técnica** (README.md):
  - Diagrama de arquitectura (incluyendo los pods, servicios y flujo de red en K8s).
  - Instrucciones paso a paso para levantar el clúster (Minikube/Kind), aplicar los manifiestos y verificar el funcionamiento.
  - Variables de entorno requeridas y cómo inyectarlas de forma segura (Secrets).
  - **Cómo configurar el /etc/hosts** para acceder al dominio conjunta3p.espe.edu.ec.

## 7. Criterios de Aceptación (Mínimos)

- El backend publica eventos en RabbitMQ para las 5 entidades principales (vinos, tiendas, usuarios, reservas, pagos).
- El microservicio consume, persiste y expone los eventos correctamente.
- El dashboard se conecta al SSE y refleja las auditorías en ≤ 2 segundos.
- Todos los servicios están contenedorizados y se despliegan en Kubernetes con un solo kubectl apply -f k8s/ (desde la carpeta k8s).
- El recurso Ingress está correctamente definido y, tras configurar el /etc/hosts, se puede acceder al dashboard y a la API de auditoría desde <http://conjunta3p.espe.edu.ec/dashboard> y <http://conjunta3p.espe.edu.ec/api/audit>.
- Los pods de auditoría escalan a 2 réplicas y el consumo de mensajes se distribuye sin duplicados.
- Los health checks (liveness/readiness) están implementados y funcionan.
- La documentación permite a otro desarrollador levantar el entorno completo en máximo 15 minutos.

## Rúbrica de Evaluación

La evaluación se realizará aplicando el comando `kubectl apply -f .` desde la carpeta /k8s del repositorio entregado. **Si el comando falla (no se aplican correctamente todos los recursos), la calificación será de 0/20 automáticamente.** El

proyecto debe ser **replicable** en cualquier computador con Kubernetes instalado (Minikube o Kind) siguiendo las instrucciones del README.

| CRITERIO                       | PUNTUACIÓN |
|--------------------------------|------------|
| RabbitMQ + Microservicio Audit | 5          |
| Implementación SSE             | 2,5        |
| Dashboard                      | 2,5        |
| Kubernetes                     |            |
| BD                             | 2,5        |
| Backend                        | 2,5        |
| Frontend                       | 2,5        |
| Ingress (Dominio local)        | 2,5        |
| <b>TOTAL</b>                   | <b>20</b>  |

### Evaluación Práctica

El docente realizará la evaluación siguiendo estos pasos:

1. Clonará el repositorio del estudiante.
2. Seguirá las instrucciones del README.md para construir las imágenes Docker (o las descargará si están en un registro público).
3. Iniciará el clúster de Kubernetes (Minikube/Kind).
4. Ejecutará `kubectl apply -f k8s/` (dentro de la carpeta k8s).
5. **Si el comando falla** (errores sintácticos, recursos no encontrados, dependencias faltantes, etc.) → **calificación 0/20**.
6. Si el despliegue es exitoso, verificará:
  - Que todos los pods estén en estado Running y con health checks pasando.
  - Que el Ingress esté activo y responda al dominio conjunta3p.espe.edu.ec (previa modificación del `/etc/hosts`).
  - Que el dashboard muestre eventos en tiempo real al realizar operaciones en el backend.
  - Que la API de auditoría devuelva datos filtrados correctamente.
  - Que el microservicio pueda escalar a 2 réplicas sin duplicar mensajes.